LSGI3245 – Geospatial Database Management and Design

Lecture 2 - Introduction to the Relational Model and SQL

Instructor: Yue YU

Department of Land Surveying and Geo-Informatics,

The Hong Kong Polytechnic University

Final Project Grouping

Assessment

Continuous Assessment + Examination

- Phase Test 1 (20%)
- Phase Test 2 (15%)
- Project (15%) (Fulfilled mostly on Lab)
- Final Examination (50%)



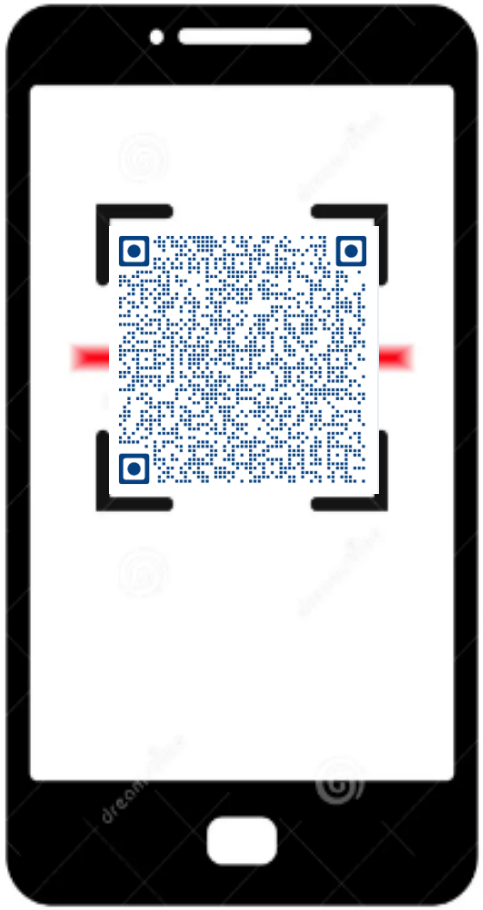
* Closed-book exam

Tentative Schedule

Week	Lecture (Tuesday)		Lab (Wednesday)	
1	1-Sept	Course Introduction	Lab Introduction and Software Installation	2-Sept
2	8-Sept	Relational Model	Introduction to PostgreSQL (Part 1)	9-Sept
3	15-Sept	SQL (Part 1)	Introduction to PostgreSQL (Part 2)	16-Sept
4	22-Sept	SQL (Part 2) Release Information for Final Project	Query and Data Manipulation Using SQL (Part 1)	23-Sept
5	29-Sept	Intermediate SQL	Query and Data Manipulation Using SQL (Part 2)	30-Sept
6	6-Oct	Database Design	Introduction to PostGIS	7-Oct
7	13-Oct	Spatial Database & PostGIS	PostGIS: Spatial Datatype and Relationships	14-Oct
8	20-Oct	Test 1	PostGIS: Spatial Queries	21-Oct
9	27-Oct	PostGIS & Spatial Queries	Students Work on Final Projects (Part 1)	28-Oct
10	3-Nov	Spatial Storage and Indexing	Students Work on Final Projects (Part 2)	4-Nov
11	10-Nov	Test 2	Students Compile Final Project Reports (Part 1)	11-Nov
12	17-Nov	Project Presentation (Part 1)	Students Compile Final Project Reports (Part 2)	18-Nov
13	24-Nov	Project Presentation (Part 2)	Summary and Outlook	25-Nov

Final Project Grouping

- 5 students per group (one as group leader)
  - Note: The assessment is based on the group as an integral part. The contributions of all the group members should be clearly stated in the oral presentation and the final written report, which are also considered for the final assessment of each group member.
- Give the group a good name
- Submit the list of group members via Google Spreadsheet
- **DDL**: by Sep 22, Tuesday (Week 4)
- If you have any queries about grouping, pls contact our TA for help: **Mr. Zhanpeng Zhang** (zhanpeng-2026.zhang@connect.polyu.hk) and **Mr. Feifan Lin** (Feifa.lin@polyu.edu.hk)



Please Scan the QR code to fill the group information

Final Project Grouping

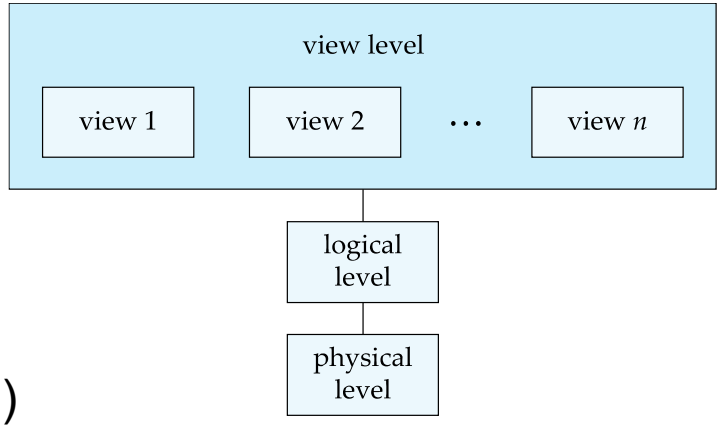
Group No	Group Name	No	Student ID	Student Name	Preferred Name (if applicable)	Group Leader
1	group name	1	group leader information			√
		2	group member information			
		3				
		4				
		5				
2		1				√
		2				
		3				
		4				
		5				
3		1				√
		2				
		3				
		4				
		5				

Fill in the grouping form via Google Spreadsheet

Recap



- Database management systems (DBMS)
  - A collection of interrelated data and a set of programs to access those data.
- The three levels of abstraction provided by the DBMS
  - Physical level
  - Logical level
  - View level
- Database languages
  - Data Definition Language (DDL)
  - Data Manipulation Language (DML)

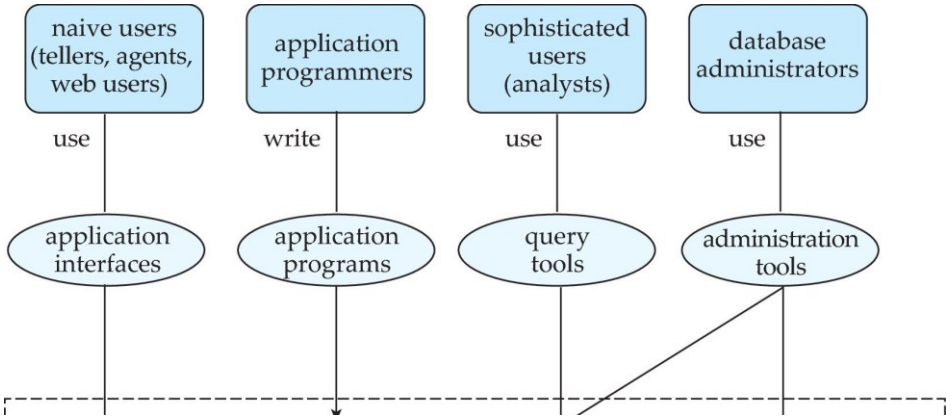


Recap (Cont.)



- Relational model & relational database
  - A relational model uses a collection of tables to represent both the data and the relationships among those data.
  - Relational model is the underlying data model for relational databases.

- Who uses databases?
  - Naive users
  - Application programmers
  - Sophisticated users
  - Database administrators



Source: http://www.db-book.com/

Learning Outcomes of Today's Class



- What is a **relational model**?

- Important concepts about relational model
  - Relation schema & instance
  - Attributes
  - Keys

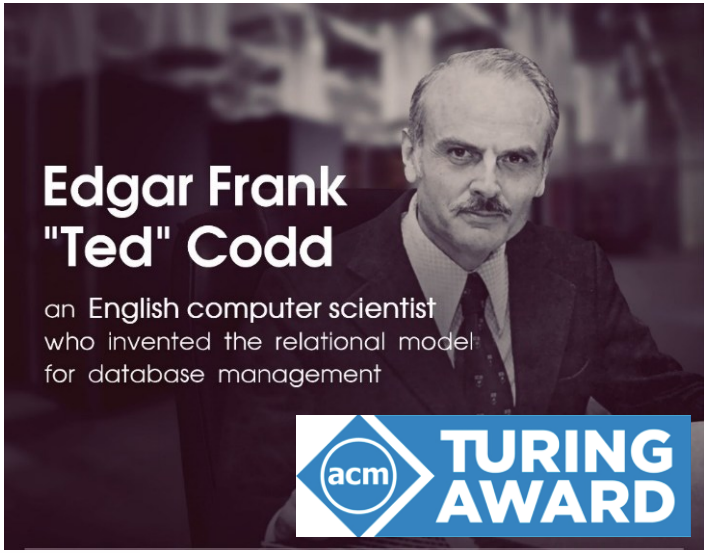
- Relational Query Languages
  - Relational algebra

- Basic syntax of SQL

The Relational Model

History

- **Edgar F. Codd** of the IBM San Jose Research Laboratory proposed the relational model in the late 1960s.
- In that paper, Codd also introduced the original definition of relational algebra.
- This work led to the prestigious ACM Turing Award to Codd in 1981.



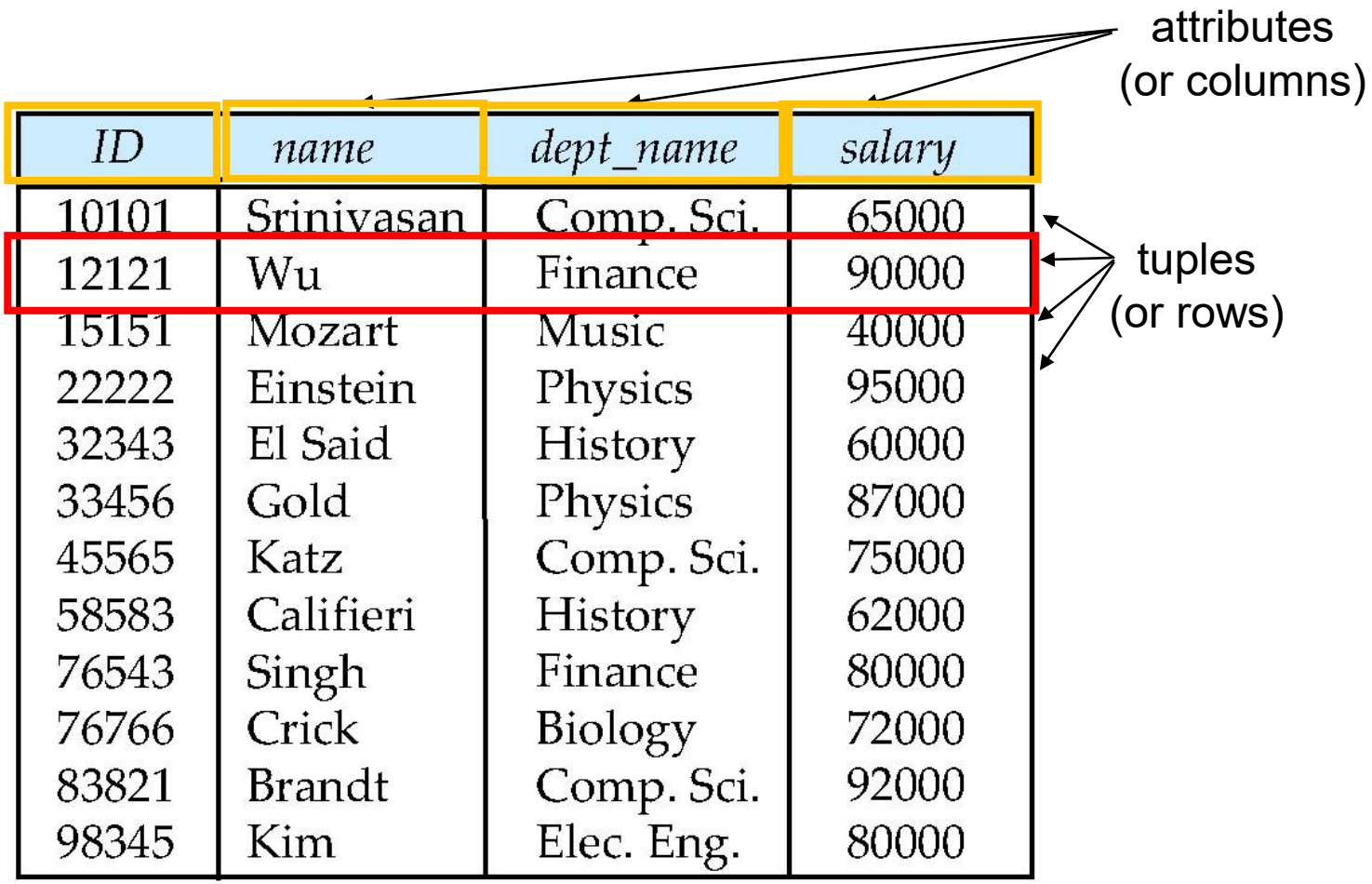
"For his fundamental and continuing contributions to the theory and practice of database management systems."

-- ACM Turing Award

Introduction to the Relational Model

- The relational model is today's primary data model for database management systems (DBMS) and commercial data-processing applications.
  - The purpose of the relational model is to provide a method for specifying data and queries.
- The relational model is based on a collection of tables. The user of the database system may query these tables, insert new tuples, delete tuples, and update tuples.
- In the relational model, the term relation is used to refer to a table.

Example of a Relation - *instructor*



ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Source: http://www.db-book.com/

Important Terms

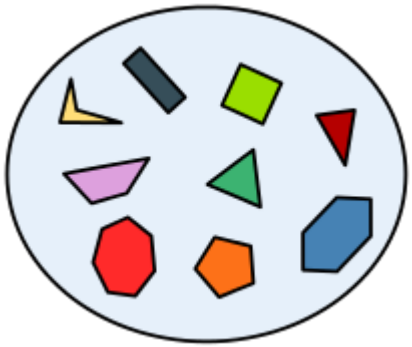
- In the relational data model, the term relation is used to refer to a table.
  - the term tuple is used to refer to a row.
  - the term attribute refers to a column of a table.
- The order in which tuples appear in a relation is irrelevant, since a relation is a set of tuples.

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Source: http://www.db-book.com/

Important Concept: Set

- In mathematics, a set is a collection of distinct objects.
- The objects that make up a set can be anything.



- The order in which the elements of a set are listed is irrelevant.

Source: https://en.wikipedia.org/wiki/Set_(mathematics)

Discussion



- How many unique sets do we have here?

$$A = \{1,2,3,4\}$$

$$B = \{1,2,2,4\}$$

$$C = \{4,3,2,1\}$$

$$D = \{1,2,4\}$$

What is the implication here?

- The order in which tuples appear in a relation is irrelevant, since a relation is a set of tuples.
- In a relation or table, we don't have duplicated rows. In other word, we won't have two rows that are exactly the same.

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Source: http://www.db-book.com/

Attributes

Domain of Attributes

- The set of allowed values for each attribute is called the domain of the attribute.

Salary Scale for Project Personnel

(effective from 1 October 2008)

Full-time

Grade	Monthly salary (HK$/month)
Professorial Project Fellow	60,000 - 78,000
Principal Project Fellow	50,000 - 65,000
Senior Project Fellow	35,000 - 52,000
Project Fellow	23,000 - 37,000
Project Associate	14,000 - 26,000
Project Assistant	10,000 - 20,000
Project Technical Assistant	9,000 - 19,000
Project Administrative Assistant	8,000 - 12,000
Student Assistant	5,000 - 8,000

Source: http://www.polyu.edu.hk/

A table for student assistants and their salaries

Name	Salary
Amy	5,000
Bob	7,500
Candy	8,000

Attribute Values

- Attribute values are (normally) required to be atomic; that is, indivisible.
- Examples:

Name	Gender	Contact
Ray	M	1234567; 2345678

  - An attribute field that records the instructor's gender (i.e., M or F) is atomic.
  - An attribute field that can record multiple phone numbers of an instructor is not atomic.
- Note: The term atomic here is different from the one used to describe database transactions.

Attribute Values (Cont.)

- The null value is a special value that signifies that the value is *unknown* or *does not exist*.

Name	Contact
Ray	*null*

- Null values can introduce challenges and complexities and should be eliminated if at all possible.
- In practice, when creating a table, we can specify that an attribute field are not allowed to take null value.

Relation Schema and Instance

Relation Schema and Instance

- A_1, A_2, …, A_n are attributes.
- R = (A_1, A_2, …, A_n) is a relation schema.
  - Example: instructor = (*ID, name, dept_name, salary*)
  - In general, ***a relation schema consists of a list of attributes and their corresponding domains***.

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Source: http://www.db-book.com/

Relation Schema and Instance

- Formally, given sets D_1, D_2, …. D_n as the domains of attributes of A_1, A_2, …, A_n, a relation r is a subset of D_1 x D_2 x … x D_n *(all the possible combinations of attribute values).*
- In other words, a relation is a set of *n*-tuples (a_1, a_2, …, a_n) where each $a_i \in D_i$.
- The current values of a relation is denoted as the relation instance.

Example

- Given two attributes A_1 and A_2, the domain of A_1 is $D_1 = \{1, 2\}$ and the domain of A_2 is $D_2 = \{4, 5, 6\}$, then $D_1 \times D_2$ refers to all the combinations of (a_1, a_2) that satisfies $a_1 \in D_1$ *and* $a_2 \in D_2$.

A_1	A_2
1	4
1	5
1	6
2	4
2	5
2	6

- An instance is a subset of the tuples (rows) in the above table.

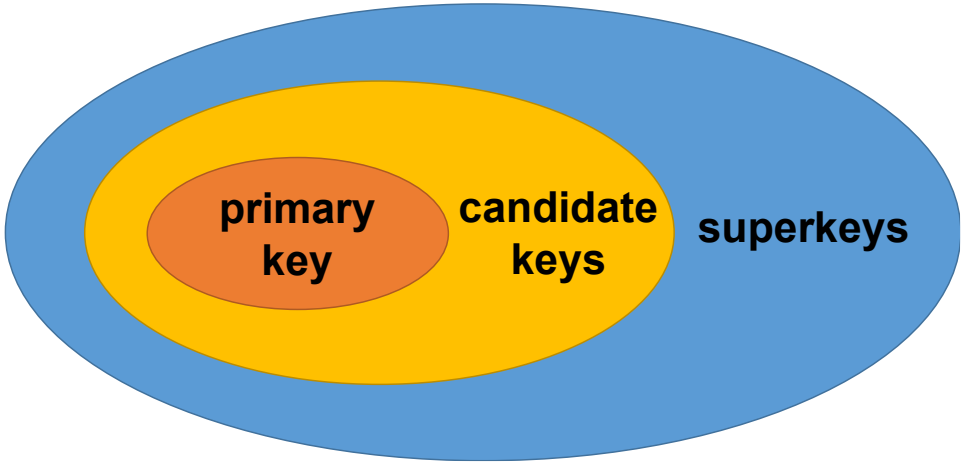
Keys



Source: Google Image

Keys

- When creating and modifying tables, we must have a way to specify how tuples within a given relation are distinguished.
- In other words, no two tuples in a relation are allowed to have exactly the same value for all attributes.



Superkey

R = (*ID, name, dept_name, salary*)

- Let K $\subseteq$ R (with R as a relation schema), *K* is a **superkey** of *R* if values for *K* are sufficient to identify a unique tuple of each possible relation *r(R).*
- In other words, a superkey is a set of one or more attributes that, take collectively, allow us to identify uniquely a tuple in a relation.

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Source: http://www.db-book.com/

Examples of K (not example of superkeys!)

- (*name, dept_name*)
- *(ID)*
- *(ID, dept_name, salary)*

R = (*ID, name, dept_name, salary*)

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Source: http://www.db-book.com/

Discussion



- A superkey is a set of one or more attributes that, take collectively, allow us to identify uniquely a tuple in a relation.
- Suppose that the attribute field ID can uniquely identify each instructor in PolyU, please list at least three superkeys of the instructor relation.

R = (*ID, name, dept_name, salary*)

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Source: http://www.db-book.com/

Answer

- Since ***ID*** is a superkey, then any K that contains the field ***ID*** is a superkey. For example:
  - *(ID, dept_name)*
  - *(ID, salary)*
  - *(ID, dept_name, salary)*
- If K is a **superkey**, then so is any superset of K.
- Examples that are NOT superkeys of the instructor relation:
  - *(dept_name)*
  - *(dept_name, salary)*

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Candidate Keys & Primary Keys

- We are often interested in superkeys for which no proper subset is a superkey. Such minimal superkeys are called **candidate keys**.
  - {*ID*} is a candidate key for ***instructor***.
  - {*ID, department*} is NOT a candidate key for ***instructor.***

- We shall use the term **primary key** to denote a candidate key that is chosen by the database designer as the principal means of identifying tuples within a relation.

Foreign Key

- Foreign key constraint: Value in one relation must appear in another
  - Referencing relation: refers to the table that contains the foreign key
  - Referenced relation: refers to the table that the foreign key is referenced to

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

  - Example: *dept_name* in ***instructor*** is a foreign key from ***instructor*** referencing ***department***
  - In this case, the instructor table is the referencing relation, and the department table is the referenced relation.

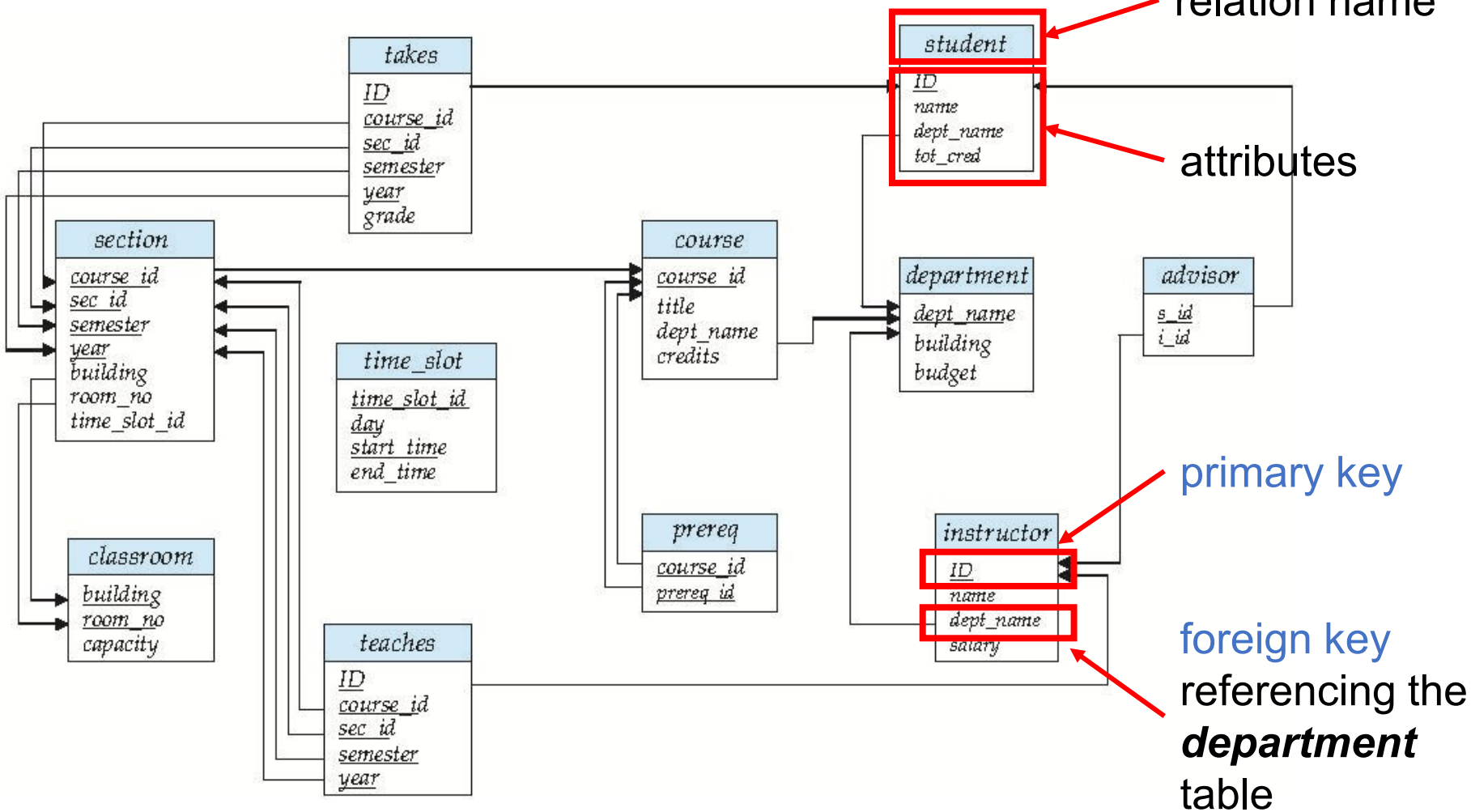
Source: http://www.db-book.com/

Schema Diagram

Schema Diagram

- A database schema, along with primary key and foreign-key constraints, can be depicted by a schema diagram.
- Each relation appears as a box, with the relation name at the top, and attributes listed inside the box.
- Primary-key attributes are shown underlined.
- Foreign-key constraints appear as arrows from the foreign-key attributes of the referencing relation to the primary key of the referenced relation.

Schema Diagram for University Database



Source: http://www.db-book.com/

Relational Query Languages

Relational Query Languages

- A query language is a language in which a user requests information from the database.
- These languages are usually on a level higher than that of a standard programming language.
- Query languages can be categorized as either procedural or nonprocedural languages.
  - In a procedural language, the user instructs the system to perform a sequence of operations on the database to compute the desired result.
  - In a nonprocedural language, the user describes the desired information without giving a specific procedure for obtaining that information.

Relational Algebra

- The relational algebra is a procedural query language which forms the theoretical basis of the SQL query language.
- The relational algebra consists of a set of operations that take one or two relations as input and produce a new relation as their result.
- These operations can be categorized as two types, i.e., unary operations and binary operations in terms of the input.

Select Operation – Selection of Rows (tuples)

the sigma sign denotes a selection operation

Relation *r*:

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$\sigma_{A=B \wedge D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10

selection criterion

The relation on which the selection is performed

Projection – Selection of Columns (Attributes)

the product sign denotes a projection operation

$\Pi_{A,C}(r)$

attribute fields involved

Relation *r*:

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2

Union of Two Relations

- It returns a relation that includes all the tuples that appear in either or both r and s.

Relation *r and s*:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r \cup s$:

A	B
α	1
α	2
β	1
β	3

Intersection of Two Relations

- It returns a relation that includes all the tuples that appear in both r and s.

Relation *r, s*:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r \cap s$

A	B
α	2

Set Difference of Two Relations

- It returns a relation that includes all the tuples that are contained in r but not in s.

Relations *r*, *s*:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

r – *s*:

A	B
α	1
β	1

Join Two Relations – Cartesian Product

- The **cartesian product** operation combines tuples from two relations, and its result contains all pairs of tuples from the two relations, regardless of whether their attribute values match.

Relation *r and s*:

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

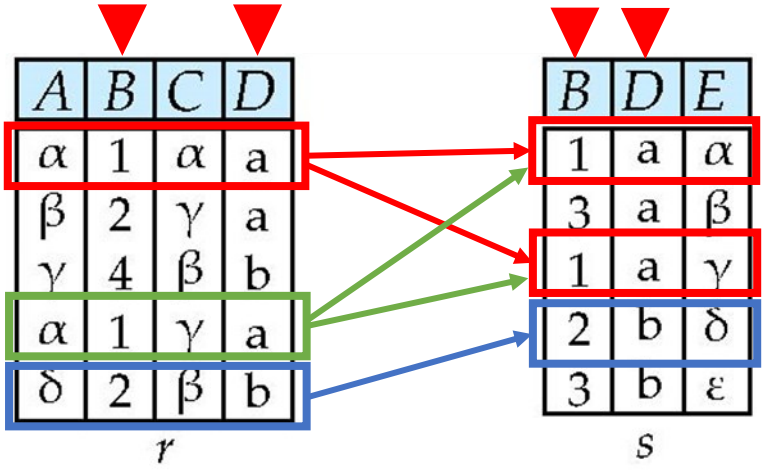
r x *s*:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Join Two Relations – Natural Join

- Different from the cartesian product operation, **natural join** will give a result that contains the combination of tuples which have the same value on all attributes that have the same name.

Relations r, s:



A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ε

s

Natural Join

$r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

$$\Pi_{A,\, r.B,\, C,\, r.D,\, E}(\sigma_{r.B = s.B \wedge r.D = s.D}(r \times s))$$

Summary of Relational Algebra Operators

Symbol (Name)	Example of Use
σ (Selection)	$\sigma_{salary >= 85000}(instructor)$
	Return rows of the input relation that satisfy the predicate.
Π (Projection)	$\Pi_{ID,\ salary}(instructor)$
	Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
x (Cartesian Product)	*instructor* x *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.
∪ (Union)	$\Pi_{name}(instructor) \cup \Pi_{name}(student)$
	Output the union of tuples from the *two* input relations.
– (Set Difference)	$\Pi_{name}(instructor) - \Pi_{name}(student)$
	Output the set difference of tuples from the two input relations.
⋈ (Natural Join)	*instructor* ⋈ *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.

Source: http://www.db-book.com/

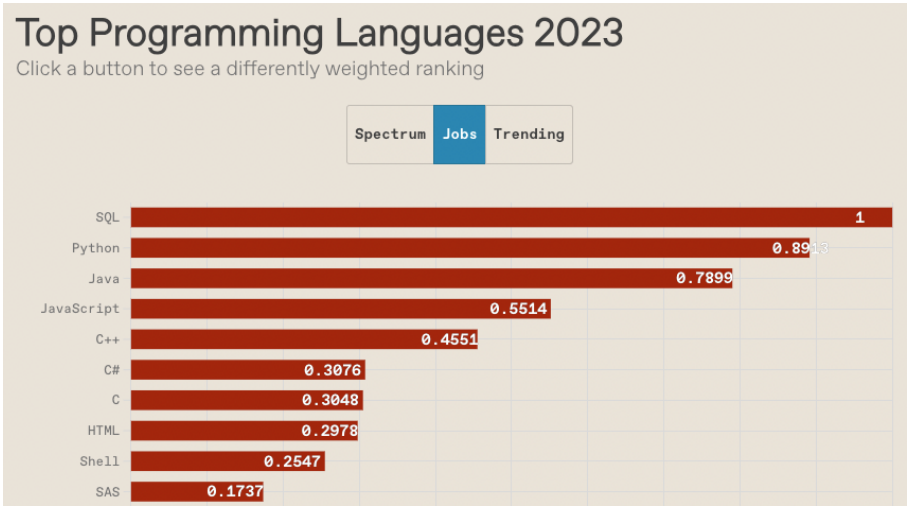
Introduction to SQL

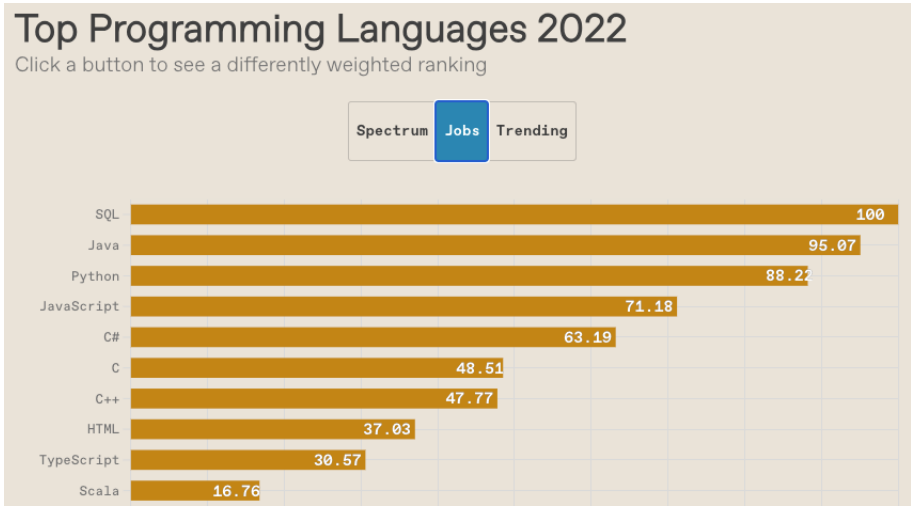
History

- SQL was initially developed at IBM by Donald D. Chamberlin and Raymond F. Boyce in the early 1970s.
- This version, initially called SEQUEL (Structured English Query Language), was designed to manipulate and retrieve data stored in IBM's original quasi-relational database management system, System R, which a group at IBM San Jose Research Laboratory had developed during the 1970s.
- The acronym SEQUEL was later changed to SQL because "SEQUEL" was a trademark of the UK-based Hawker Siddeley aircraft company.
- How to pronounce SQL?
  - S-Q-L ('Ess-cue-ell')
  - Sequel

SQL in High Demand

- In the IEEE Specturm's annual rankings of the Top Programming Languages 2022 and 2023, under the "Jobs" ranking, SQL ranks No. 1.





- The employers prefer to hire employees with SQL skills in tandem with some other language such as Java or C++ (a given language *plus* SQL).

SQL in GIS Software

SQL in ArcGIS

- Queries are used to search for and select a subset of features and table records
- All query expressions in ArcGIS use SQL to formulate these search specifications



SQL in QGIS

- The Query Builder allows you to define a subset of a table using a SQL-like WHERE clause and to display the result in the main window.
- The query result can then be saved as a new vector layer



Source: ArcGIS & QGIS

Major Components of SQL

- **Data-definition language (DDL):** provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- **Data-manipulation language (DML):** to insert tuples into, delete tuples from, and modify tuples in the database.
- **Integrity:** specifies integrity constraints.
- **View definition:** defines views.
- **Transaction control:** specifies the beginning and ending of transactions.
- **Embedded SQL** and **dynamic SQL:** define how SQL statements can be embedded within general-purpose programming languages.
- **Authorization:** specifies access rights to relations and views.

How to define a schema for a table?

Create A Table

- An SQL relation is defined using the **create table** command:

create table r (A_1 D_1, A_2 D_2, ..., A_n D_n,
(integrity-constraint$_1$),
...,
(integrity-constraint$_k$))

  - r is the name of the relation
  - each A_i is an attribute name in the schema of relation r
  - D_i is the data type of values in the domain of attribute A_i

- Example:

create table *instructor* (
ID **char**(5),
name **varchar**(20),
dept_name **varchar**(20),
salary **numeric**(8,2))

Domain Types in SQL

- **char(n)**. Fixed length character string, with user-specified length n.
- **varchar(n)**. Variable length character strings, with user-specified maximum length n.
- **int**. Integer (a finite subset of the integers that is machine-dependent).
- **smallint**. Small integer (a machine-dependent subset of the integer domain type).
- **numeric(p,d)**. Fixed point number, with user-specified precision of p digits, with d digits to the right of decimal point. (ex., numeric(3,1), allows 44.5 to be stores exactly, but not 444.5 or 0.32)
- **real, double precision**. Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(n)**. Floating point number, with user-specified precision of at least n digits.

Example: PostgreSQL



Chapter 8. Data Types

Table of Contents

8.1. Numeric Types
- 8.1.1. Integer Types
- 8.1.2. Arbitrary Precision Numbers
- 8.1.3. Floating-Point Types
- 8.1.4. Serial Types

8.2. Monetary Types

8.3. Character Types

8.4. Binary Data Types
- 8.4.1. bytea Hex Format
- 8.4.2. bytea Escape Format

8.5. Date/Time Types
- 8.5.1. Date/Time Input
- 8.5.2. Date/Time Output
- 8.5.3. Time Zones
- 8.5.4. Interval Input
- 8.5.5. Interval Output

8.6. Boolean Type

Table 8.2. Numeric Types

Name	Storage Size	Description	Range
smallint	2 bytes	small-range integer	-32768 to +32767
integer	4 bytes	typical choice for integer	-2147483648 to +2147483647
bigint	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
decimal	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
numeric	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
real	4 bytes	variable-precision, inexact	6 decimal digits precision
double precision	8 bytes	variable-precision, inexact	15 decimal digits precision
smallserial	2 bytes	small autoincrementing integer	1 to 32767
serial	4 bytes	autoincrementing integer	1 to 2147483647
bigserial	8 bytes	large autoincrementing integer	1 to 9223372036854775807

Table 8.4. Character Types

Name	Description
character varying(*n*), varchar(*n*)	variable-length with limit
character(*n*), char(*n*), bpchar(*n*)	fixed-length, blank padded
text	variable unlimited length

https://www.postgresql.org/docs/current/datatype.html

Integrity Constraints in Create Table

- **not null**
- **primary key** (A_1, ..., A_n)
- **foreign key** (A_m, ..., A_n) **references** r

- Example:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

```
create table instructor (
    ID          char(5),
    name        varchar(20) not null,
    dept_name   varchar(20),
    salary      numeric(8,2),
    primary key (ID),
    foreign key (dept_name) references department);
```

primary key declaration on an attribute automatically ensures **not null**

Updates to Tables

- **Insert**
  - **insert into** *instructor* **values** (‘10211’, ’Smith’, ’Biology’, 66000);
- **Delete**
  - **delete** from *student*
- **Drop** Table
  - **drop** table *r*
- **Alter**
  - **alter** table *r* **add** A D
  - **alter** table *r* **drop** A

How to perform queries?

Basic Query Structure

- A typical SQL query has the form:

select $A_1, A_2, \ldots, A_n$
from $r_1, r_2, \ldots, r_m$
where P

  - A_i represents an attribute
  - r_i represents a relation
  - P is a predicate.
- The result of an SQL query is a relation.

The *Select* Clause

- The select clause lists the attributes desired in the result of a query.
  - An asterisk (*) in the select clause denotes “all attributes”.
- Example:

select *name*
from *instructor;*

select *
from *instructor*

- SQL names are case insensitive (i.e., you may use upper- or lower-case letters.)
  - *E.g., Name = NAME = name*

What you will get…

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Message

	name character varying (20)
1	Srinivasan
2	Wu
3	Mozart
4	Einstein
5	El Said
6	Gold
7	Katz
8	Califieri
9	Singh
10	Crick
11	Brandt
12	Kim

select *name*
from *instructor;*

The *Select* Clause (Cont)

- SQL allows duplicates in relations as well as in query results.
- To force the elimination of duplicates, insert the keyword **distinct** after select.
- Find the department names of all instructors, and remove duplicates:

```
select distinct dept_name
from instructor
```

- The keyword **all** specifies that duplicates should not be removed.

```
select all dept_name
from instructor
```

What you will get…

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Messag

	dept_name character varying (20)
1	Biology
2	Physics
3	Finance
4	Music
5	History
6	Elec. Eng.
7	Comp. Sci.

select distinct *dept_name*
from *instructor*

The *Select* Clause (Cont)

- The **select** clause can contain arithmetic expressions involving the operation, +, –, *, and /, and operating on constants or attributes of tuples.

select *ID, name, salary/12*
from *instructor*

  - The result would return a relation that is the same as the *instructor* relation, except that the value of the attribute salary is divided by 12.
  - Can rename "salary/12" using the as clause:

select *ID, name, salary/12*
as *monthly_salary*
from *instructor*

What you will get…

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	monthly_salary numeric
1	10101	Srinivasan	5416.6666666666666667
2	12121	Wu	7500.0000000000000000
3	15151	Mozart	3333.3333333333333333
4	22222	Einstein	7916.6666666666666667
5	32343	El Said	5000.0000000000000000
6	33456	Gold	7250.0000000000000000
7	45565	Katz	6250.0000000000000000
8	58583	Califieri	5166.6666666666666667
9	76543	Singh	6666.6666666666666667
10	76766	Crick	6000.0000000000000000
11	83821	Brandt	7666.6666666666666667
12	98345	Kim	6666.6666666666666667

select *ID, name, salary/12*
as *monthly_salary*
from *instructor*

The *where* Clause

- The **where** clause allows us to select only those rows in the result relation of the from clause that satisfy a specified predicate.

select *name*
from *instructor*
where *dept_name = 'Comp.Sci'*

- Comparison results can be combined using the logical connectives **and, or,** and **not**

select *name*
from *instructor*
where *dept_name = 'Comp.Sci'* **and** *salary > 80000*

What you will get…

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Messag

	name character varying (20)
1	Brandt

select *name*
from *instructor*
where *dept_name = 'Comp.Sci'* **and** *salary > 80000*

The *from* Clause

- The **from** clause lists the relations involved in the query
  - Corresponds to the Cartesian product operation of the relational algebra.
- Find the Cartesian product instructor × teaches.

```
select *
from instructor, teaches
```

  - Generates every possible instructor-teaches pair, with all attributes from both relations.
  - For common attributes (e.g., *ID*), the attributes in the resulting table are renamed using the relation name (e.g., *instructor.ID*)
- Cartesian product not very useful directly, but useful combined with where-clause condition (selection operation in relational algebra).

Example

select *
from *instructor, teaches*

instructor

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

teaches

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
				2009

Inst.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2009
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010
12121	Wu	Pinance	90000	10101	CS-347	1	Fall	2009
12121	Wu	Pinance	90000	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2010
12121	Wu	Pinance	90000	22222	PHY-101	1	Fall	2009
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…

Combining *from* and *where* clause

- Find the names of all instructors who have taught some course and the course_id

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID*

- Find the names of all instructors in the Art department who have taught some course and the course_id.

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID* **and** *instructor.dept_name* = 'Art'

Example

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID*

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

teaches

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
				2009

Inst.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2009
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010
12121	Wu	Pinance	90000	10101	CS-347	1	Fall	2009
12121	Wu	Pinance	90000	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2010
12121	Wu	Pinance	90000	22222	PHY-101	1	Fall	2009
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…

Summary



- What is a **relational model**?
  - The relational model is based on a collection of tables. The user of the database system may query these tables, insert new tuples, delete tuples, and update tuples.
  - In the relational model, the term relation is used to refer to a table.

- Important concepts about relational model
  - Relation schema & instance
  - Attributes
  - Keys

Summary



- Relational Query Languages
  - Relational algebra
- Basic syntax of SQL
  - create table
  - select
  - from
  - where

Symbol (Name)	Example of Use
σ (Selection)	$\sigma_{salary >= 85000}(instructor)$
	Return rows of the input relation that satisfy the predicate.
Π (Projection)	$\Pi_{ID,\ salary}(instructor)$
	Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
x (Cartesian Product)	*instructor* x *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.
∪ (Union)	$\Pi_{name}(instructor) \cup \Pi_{name}(student)$
	Output the union of tuples from the *two* input relations.
– (Set Difference)	$\Pi_{name}(instructor) - \Pi_{name}(student)$
	Output the set difference of tuples from the two input relations.
⋈ (Natural Join)	*instructor* ⋈ *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.

Source: http://www.db-book.com/



Thank you!

michael-yue.yu@polyu.edu.hk

Source: Google Image